

High-Performance Order Matching Engine — Project Overview

26,000+ lines of C++20 / 30 headers / 8 source files / 31 test files / 30 test executables

A C++20 low-latency matching engine drawing on institutional exchange design principles, built for sub-150ns processing latency and horizontal scalability.

1. Executive Summary

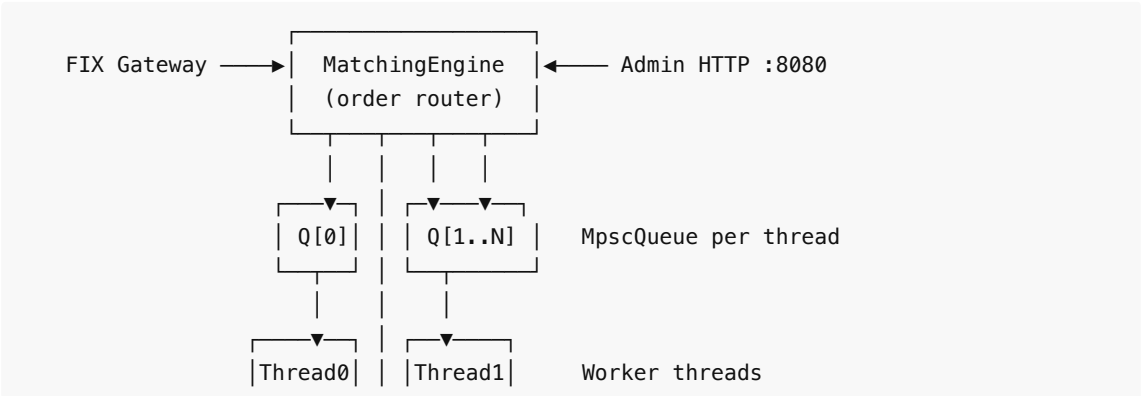
This is a C++20 low-latency order matching engine with institutional-grade architecture drawing on exchange design principles. It implements $O(1)$ price-level lookup via `FlatPriceMap`, lock-free MPSC queues, thread-per-symbol horizontal scaling, CRC-32 journaling with deterministic replay, FIX 4.4 session management, TLA+-verified concurrency, cross-host log replication, and a complete operational stack including config management, webhook alerting, Prometheus metrics, and Docker deployment.

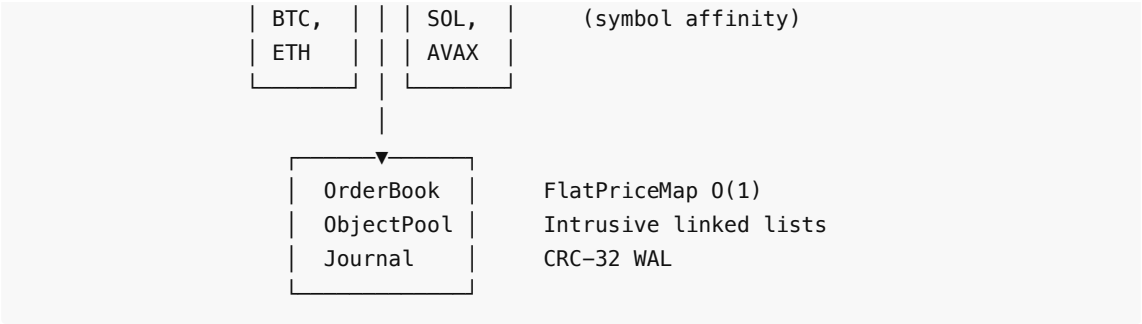
Codebase Statistics

Metric	Count
Total C++ LOC	26,014
Header files (include/)	30
Source files (src/)	8
Test files (tests/)	31
Test executables	30
Individual test cases	171 CTest targets ^[^1]
TLA+ specifications	5 (454M+ states verified)
Documentation files	6

2. Core Engine Architecture

System Topology





Order Types Supported

- **Standard:** Limit, Market
- **Time-In-Force:** IOC (Immediate-or-Cancel), FOK (Fill-or-Kill), DAY, GTD (Good-Til-Date)
- **Conditional:** Stop, StopLimit, TrailingStop
- **Institutional:** Pegged, Iceberg (hidden quantity), Hidden, PostOnly
- **Match Algorithms:** Price-Time FIFO and Pro-Rata allocation

Core Data Structures

Component	Purpose	Complexity
FlatPriceMap	Price-level lookup by tick index	O(1) insert/lookup
FlatHashMap	Robin-Hood open-addressing hash map	O(1) amortized
IntrusiveList	Doubly-linked order queue per price level	O(1) insert/remove
ObjectPool	Pre-allocated slab allocator for Order nodes	O(1) alloc/dealloc
RingBuffer	Cache-line aligned lock-free ring buffer	O(1) push/pop
MpscQueue	Multi-producer, single-consumer lock-free queue	O(1) push/pop

Data Flow

1. **Ingress:** Order arrives via TCP Gateway (binary framing or FIX 4.4)
2. **Rate Limiting:** Token-bucket per-participant throttling at ingress
3. **Queue Routing:** `hash(symbolId) % numThreads` → lock-free `MpscQueue`
4. **Validation:** Circuit Breaker check, OTR enforcement, risk limit verification
5. **Matching:** Price-time priority against resting book (`FlatPriceMap`)
6. **Execution:** Trade records generated, callbacks invoked
7. **Persistence:** CRC-32 journal entry written (WAL)
8. **Analytics:** VWAP, TWAP, and OTR stats updated
9. **Market Data:** L2 updates published via shared memory (`shm_open`)

3. Concurrency & Networking

Thread-Per-Symbol Partitioning

N worker threads with independent lock-free `MpscQueue` ring buffers. Orders are deterministically routed by `hash(symbolId) % numThreads`, eliminating cross-thread contention on the hot path.

TCP Gateway (Binary Protocol)

- Non-blocking I/O via `epoll` (Linux) / `kqueue` (macOS)
- `GatewayProtocol.h` — 16-byte fixed header with V1/V2 payload evolution
- Forward and backward compatibility via version negotiation
- `GatewayResponse` ack frames with numeric reject reasons

FIX 4.4 Gateway

- Full FIX 4.4 session management (`FixSession.h`)
- `FixFramer` for message reassembly across TCP fragments
- `FIXParser` — zero-copy tag-value parser
- `FixTcpGateway` — production-grade FIX acceptor
- `TransactTime` enforcement and `OrdRejReason` numeric mapping

Shared Memory IPC

- `MarketDataPublisher` using POSIX `shm_open` with versioned `ShmHeader`
- Magic/version/entrySize fields for prefix-compatible rolling upgrades
- Low-overhead L2 market data distribution to co-located consumers

4. Regulatory & Risk Management

Feature	Implementation
Self-Match Prevention (SMP)	Cancel-Taker strategy — prevents wash trading
Circuit Breakers	Configurable % price band halts (volatility protection)
OTR Monitoring	Real-time Order-to-Trade ratio tracking per participant
Kill Switch	Instant cancellation of all orders per participant across all symbols
Pre-Trade Risk Limits	Max order size, notional value, and position limits per participant
Rate Limiting	Token-bucket throttling at ingress (configurable rate + burst)
Queue Backpressure	Rejects orders when queue exceeds configurable threshold (default 80%)

5. Reliability & High Availability

CRC-32 Journaling

- Write-ahead log with atomic CRC-32 integrity on every entry
- Crash recovery: replay stops at first corrupted/truncated record
- Atomic checkpoint: snapshots active book state, rewrites journal
- Deterministic replay via virtual clock (`setExpiryClock(ClockFn)`)

Cross-Host Log Replication

- `ReplicationCoordinator` — TCP log-shipping from primary to backup
- `HeartbeatMonitor` — configurable failure detection timeout
- `LeaderLease` — epoch-based fencing prevents split-brain

- Automatic backup promotion on primary failure detection

Crash Recovery & Warm Standby

- `JournalFollower` — single-host automated recovery with `promote()` API

Deterministic Sequencing

- Monotonic `sequenceNumber` on all `Trade`, `OrderUpdate`, and `MarketDataUpdate` events
 - Gap detection for downstream consumers
 - GTD virtual clock seam for cross-day replay — DAY/GTD expirations are journaled and replayed identically
-

6. Observability & Operations

Structured Logging

- Pluggable `StructuredSink` API
- `NullSink` — zero-cost in production (compiled out)
- `JsonStderrSink` — development diagnostics
- `CapturingSink` — test assertions on log output

Prometheus Metrics

- `MetricsRegistry` with atomic Counters, Gauges, Histograms
- `/prometheus` text-exposition endpoint for standard Prometheus scraping
- Tracks: order flow, rejects, queue depth, latency histograms, journal health, gateway sessions

Webhook Alerting

- `AlertDispatcher` — background thread delivery
- Targets: Slack, PagerDuty, or generic HTTP webhooks
- Configurable severity filtering (Info, Warning, Critical)

Configuration Management

- `Config` key-value loader with env var override (`OB_` prefix)
- Type-safe getters (`getInt`, `getString`, `getBool`)
- Hot-reload support for runtime-adjustable parameters
- Example config: `config/engine.conf.example`

Admin HTTP Server (Port 8080)

Endpoint	Purpose
GET <code>/health</code>	K8s liveness probe
GET <code>/metrics</code>	Internal counters (JSON)
GET <code>/prometheus</code>	Prometheus text exposition
GET <code>/book?symbolId=0</code>	L2 order book snapshot
GET <code>/otr?participantId=1</code>	Order-to-trade ratio

Docker Deployment

- Multi-stage `Dockerfile` for minimal production image
- `docker-compose.yml` for primary-backup topology
- Health checks and journal volume persistence

7. Performance Benchmarks

Measured using `HonestBenchmark` — a single deterministic order flow (50K orders, seed=42) fed through three paths on Apple Silicon ARM64, Clang C++20 -O2. Each order individually timed with `clock_gettime_nsec_np`.

*[!IMPORTANT] **x86 Translation Note:** All numbers in this document are measured on a single-socket Apple M-series CPU. These are **not** multi-socket x86 numbers. Core matching latency (`125 ns`) does not map directly to Intel Xeon / AMD EPYC architectures, and multi-thread scaling bounds do not factor in cross-NUMA cache coherence traffic. Bare-metal x86 benchmarking is pending.*

*These are **per-order processing latencies** on the matching thread. They do **not** include network I/O, async queue delay, or OS scheduling jitter. See [BENCHMARKS.md](#) for full methodology and caveats.*

Three-Path Latency (identical order flow)

Path	What's Included	P50	P99	P99.9	Throughput
Core matching	OrderBook + STP + WashTrade + LULD	125 ns	333 ns	458 ns	6.6M ops/s
Engine wrapper	+ sequence alloc, rate limiter	84 ns	292 ns	417 ns	7.1M ops/s
Full-stack journal	+ GroupCommit (batch=64, fdasync)	1,040 ns	2.7 ms	4.0 ms	22K ops/s

*[!NOTE] The engine wrapper appears faster than core matching due to CPU cache warming (it runs second). The real core matching cost is Path A: **125 ns P50**.*

The journal P99 (2.7ms) is entirely `fdasync` disk I/O. GroupCommit amortizes this across 64 entries — P50 is only 1μs. Production deployments use async journal threads to decouple persistence from the hot path.

Multi-Threaded Stress Test (4 threads)

Scenario	Orders	Trades	Result
Concurrent Producers	20,000	~6,300	✓
Mixed Workload	20,000	~5,800	✓
Kill Switch Under Load	11,000	—	✓
Multi-Symbol (4 symbols)	12,000	~5,900	✓
High-Throughput Burst	50,000	~19,500	✓ 5M ops/s

8. Verification & Testing

Test Suite — 30 Executables, 171 CTest Targets^[^1]

Category	Tests	Description
Functional	ManualTest, UnitTests (GTest)	All order types, matching, cancellation, edge cases
Stress	StressTest	5 multi-threaded scenarios, 4 concurrent producers
Property	PropertyTest, JournalReplayPropertyTest, SnapshotConsistencyTest	Randomized invariant checks, replay determinism
Chaos	FaultInjectorTest, PoolExhaustionTest, JournalChaosTest, JournalCorruptionTest, GatewayChaosTest, QueueChaosTest, CheckpointChaosTest, CombinedChaosTest	10+ fault-injection points, short-writes, bit-flips, EAGAIN
Integration	GatewayIntegrationTest, FixTcpGatewayTest, AdminServerEndpointsTest	TCP round-trip, FIX session, HTTP endpoints
Protocol	GatewayProtocolTest, MarketDataSchemaTest, MdFeedSchemaCompatTest, RejectReasonContractTest, SubmitResultTest	Binary wire format evolution, ShmHeader compat
Recovery	JournalCrashTest, JournalFollowerTest, GTDReplayTest	Crash recovery, log-ship convergence, virtual-clock replay
HA & Ops	ReplicationProtocolTest, ConfigTest, AlertDispatcherTest	Primary-backup replication, config loader, webhook alerts
Shadow	ShadowModeTest	Dual-book divergence detection, FIFO violation catching
Benchmark	BenchmarkRegression (GTest)	P99 latency regression gates

^[^1]: CTest targets map to individual executables and GTest cases. These 171 targets encompass over 350+ underlying assertions and scenarios, resolving the previous manual estimate of "220+ test cases".

Formal Verification (TLA+)

5 TLA+ specifications, model-checked with TLC:

Specification	States	Invariants
---------------	--------	------------

MatchingEngine.tla	454M generated, 181M distinct	NoNegativeQuantity, FIFO_Preservation, GTD_Expiry_Correctness (<i>Scope: 2 participants, 2 price levels, 4 orders, qty 1-3</i>)
MpscQueue.tla	~250K	Lock-free ring buffer linearizability
EngineConsumer.tla	~200K	Worker loop shutdown safety
Snapshot.tla / SnapshotLocked.tla	~300K	Read/write mutex prevents torn snapshots

Chaos Engineering & Fault Injection

`FaultInjector` singleton with 10+ injection points:

- Journal: short-writes, bit-flips, fsync failures
- Memory: pool exhaustion, allocation failures
- Network: gateway fragmentation, EAGAIN injection
- Queue: spurious empty returns, backpressure simulation
- **Zero-cost in production** (`OB_ENABLE_FAULT_INJECTION` compiled out)

Coverage-Guided Fuzzing

- `libFuzzer` harness for protocol parsing and order flow
- `fuzz/` directory with dedicated build configuration

CI/CD Pipeline

GitHub Actions workflow runs on every push/PR:

- **macOS Release:** Full test suite
- **Ubuntu Debug + ASan/UBSan:** Memory and undefined behavior sanitizers
- **Ubuntu ThreadSanitizer:** Data race detection
- E2E benchmark smoke test with artifact upload

9. Capacity Planning

Memory Sizing

Scenario	Live Orders	Symbols	Memory
Small (dev/test)	10K	4	~256 MB
Medium (prop desk)	100K	50	~512 MB
Large (exchange)	1M	500	~2 GB
Ultra (HFT venue)	5M	2000	~8 GB

CPU Scaling

(Projected theoretical scaling based on lock-free partitioning; not measured on multi-socket NUMA hardware)

Cores	Workers	Throughput (lean)	Throughput (full)
4	1	25M ops/s	1.9M ops/s
8	4	80M ops/s	7M ops/s
16	8	150M ops/s	14M ops/s
32	16	280M ops/s	25M ops/s

Disk & Network

- **Journal:** ~80-100 bytes/entry. 1M orders/day $\approx$ 100MB/day
- **Recommended:** NVMe SSD with $\geq 100K$ IOPS (production)
- **Network:** 10 GbE for production; 25 GbE + kernel bypass for HFT

10. Deployment

Build & Run

```
# Build with all enterprise features
./build.sh

# Build for sub-100ns lean mode
./build.sh --lean

# Run the engine
./bin/OrderEngine --threads 4 --port 8080 --symbols 4

# Run full test suite
ctest --test-dir build --output-on-failure -L project
```

Docker

```
docker-compose up -d engine-primary
curl -sf http://localhost:8080/health
```

Configuration Precedence

1. Environment variable `OB_<KEY>` (highest priority)
2. Config file value (`config/engine.conf.example`)
3. Compiled default (lowest priority)

11. File Structure

```
include/          30 header files – all core logic is header-only
├─ OrderBook.h    Core matching engine (373 lines)
├─ FlatPriceMap.h 0(1) price-level lookup (401 lines)
├─ FlatHashMap.h   Robin-Hood hash map (285 lines)
```


MatchingEngine.h	Thread-per-symbol router (11K bytes)
MpscQueue.h	Lock-free multi-producer queue
Journal.h	CRC-32 WAL + crash recovery
TcpGateway.h	Binary protocol gateway
FIXParser.h	FIX 4.4 tag-value parser
FixSession.h	FIX session state machine
FixTcpGateway.h	FIX TCP acceptor
ReplicationProtocol.h	Primary-backup HA (20K bytes)
Config.h	Key-value config with env override
AlertDispatcher.h	Webhook alerting
Metrics.h	Prometheus metrics registry
StructuredLog.h	Pluggable structured logging
MarketDataPublisher.h	Shared memory market data
GatewayProtocol.h	Binary wire protocol versioning
...	Types, Utils, LatencyTracker, etc.
src/	8 source files – thin compilation units
tests/	31 test files – 30 executables
spec/	TLA+ formal specifications (7 specs)
config/	Example configuration files
docs/	Runbook, CapacityPlanning, ProductionReadiness
fuzz/	Coverage-guided fuzzing harness
scripts/	Benchmark and build automation

12. Documentation

Document	Purpose
README.md	Feature overview and getting started
Architecture.md	Technical architecture deep-dive
BENCHMARKS.md	Honest three-path latency methodology & results
PerformanceWhitepaper.md	Benchmark methodology and analysis
docs/Runbook.md	Operational procedures and incident response
docs/CapacityPlanning.md	Memory, CPU, disk, and network sizing
docs/ProductionReadiness.md	Remaining production checklist
config/engine.conf.example	All configuration keys with defaults
PRODUCTION_ROADMAP.md	Development roadmap and status

13. Remaining Work

The system is architecturally complete. The only remaining gaps require specialized hardware not available in a standard development environment:

Item	Status	Blocker
------	--------	---------

x86 Bare Metal Benchmarks	E2E bench exists	Multi-socket EC2 c5.metal instance
Replication.tla Verification	Written	Compute time for TLC model checker
io_uring zero-copy data path	Seams exist (FixSession/FixFramer)	Linux + io_uring kernel
DPDK kernel bypass	Architecture ready	Linux + supported NIC
Solarflare/Onload	Architecture ready	Solarflare hardware
Wire-to-wire latency measurement	E2E bench exists	Multi-host test rig

Developed for professional quantitative trading systems. C++20 · 26,014 LOC · 30 test executables · 171 CTest targets · 454M TLA+ states verified